Copyright All rights reserved

04. 연산자 (Operator)

Heesung Oh
https://github.com/3dapi

4.0 연산자

Heesung Oh
Copyright All rights reserved

- 연산자
  - 값 계산, 비교, 논리 판단, 대입 등 연산 표현하는 문법적 기호
  - 하나 이상의 피연산자(operand)와 함께 사용하여 연산 수행
  - 연산 결과로 새로운 값 생성 또는 변수 값 변경

4.1 대입 연산자와 변수 복사

Heesung Oh
Copyright All rights reserved

- 대입 연산자 (assignment operator: =)
  - 오른쪽 표현식의 결과를 왼쪽 변수에 저장
  - 값 전달 방향 오른쪽 → 왼쪽
  - 저장 형식 왼쪽 변수의 자료형이 결정
  - 복합 대입 연산자(+=, -=, *=, /= 등)의 기반

- 특징
  - 자료형이 다르면 암묵적 형 변환 발생
  - 대입식은 표현식(expression)
  - 연쇄 대입 가능

4.1 대입 연산자와 변수 복사 예

Heesung Oh
Copyright All rights reserved

- 변수에 값 할당 (저장)

```
int a;
a = 10;      // 변수 a에 10 저장
```

- 타입 변경으로 데이터 유실이 발생할 수 있으면 경고

```
float f = 3.7f;
int i;
i = f;
```

- 표현식

```
int a, b;
a = (b = 5); // 5 → b→ a
```

4.2 쉼표 연산자 (comma operator: ,)

Heesung Oh
Copyright All rights reserved

- 쉼표 연산자(Comma operator)
  - 여러 표현식을 순서대로 평가(evaluate)하는 연산자
  - 왼쪽 → 오른쪽 순서로 실행
  - 마지막 표현식의 결과 반환

```
int a;
a = (1, 2, 3);   // 1, 2 평가, 3을 a에 대입
```

4.2 쉼표 연산자 예

Heesung Oh
Copyright All rights reserved

```
int x = 0, y = 0;
x = (y = 10, y + 5);
// y = 10 실행 → y + 5 실행 → 결과 15가 x에 저장
```

- for loop에서 쉼표 연산자

```
for (i = 0, j = 10; i < j; i++, j--) {
    // ...
}
```

4.3 산술 연산자

Heesung Oh
Copyright All rights reserved

- 산술 연산자 (arithmetic operator: +, −, *, /, %)
  - 숫자 데이터를 계산하기 위한 기본 연산자
  - 정수형 또는 실수형 데이터에 사용
  - 덧셈, 뺄셈, 곱셈, 나눗셈, 나머지 연산 수행
  - %는 정수형에만 사용

- 특징
  - 자료형에 따라 계산 방식 달라짐
  - 정수 연산과 실수 연산 결과 차이
  - 우선순위: *, / → +, −

Heesung Oh
Copyright All rights reserved

- 산술 연산자 요약

연산자	이름	의미	예 (x:100, y:7)	결과
+	덧셈 연산자	더하기	x + y	107
-	뺄셈 연산자	빼기	x - y	93
*	곱셈 연산자	곱하기	x * y	700
/	나눗셈 연산자	나누기	x / y	14
%	모듈로(modulo), 나머지 연산자	정수 나눗셈 나머지	x % y	2

- 활용 예
  - 짝수/홀수 판별: n % 2 == 0
  - 배수 판별: n % k == 0
  - 순환 인덱스(wrap-around): (i + 1) % size
    ⇐ // 0~size 사이를 순환

4.3 산술 연산자 – 예

Heesung Oh
Copyright All rights reserved

- **예: 산술 연산자**

```
int a = 10 + 3;    // 13
int b = 10 - 3;    // 7
int c = 10 * 3;    // 30
int d = 10 / 3;    // 3 (정수 나눗셈 → 소수점 버림)
int e = 10 % 3;    // 1 (모듈로 연산)
```

- **실수형은 불가능**

```
5.0f % 2.0f;    // 오류
```

- **우선 순위**

```
int v = 2 + 3 * 4;    // 3 * 4 먼저 계산 → 2 + 12 = 14
```

4.4 관계 연산자

Heesung Oh
Copyright All rights reserved

- relational operator: ==, !=, 〉, 〉=, 〈, 〈=
  - 두 값을 비교 참(true) 또는 거짓(false) 반환
  - 연산의 결과는 정수형(int)
    - 참(true): 1
    - 거짓(false): 0

- 특징
  - 조건 판단에 사용
  - 논리 연산자(&&, ||, !)와 함께 복합 조건식 구성
  - 우선순위: 산술 연산자 〉 관계 연산자 〉 논리 연산자

4.4 관계 연산자

Heesung Oh
Copyright All rights reserved

- 의미

연산자	의미	설명	예 (x:100, y:7)	결과
==	같음	두 값이 같으면 참	x == y	0 (false)
!=	같지 않음	두 값이 다르면 참	x != y	1 (true)
>	크다	왼쪽 값이 더 크면 참	x > y	1 (true)
>=	크거나 같다	왼쪽 값이 크거나 같으면 참	x >= y	0 (false)
<	작다	왼쪽 값이 더 작으면 참	x < y	0 (false)
<=	작거나 같다	왼쪽 값이 작거나 같으면 참	x <= y	0 (false)

4.4 관계 연산자 – 예

Heesung Oh
Copyright All rights reserved

```c
int a = 10, b = 20;

int r1 = (a == b);  // 0 (거짓)
int r2 = (a != b);  // 1 (참)
int r3 = (a < b);   // 1 (참)
int r4 = (a >= 10); // 1 (참)
```

```c
// 1) 조건문(if)
if (score >= 90) {
    printf("A 등급\n");
}

// 2) 반복문(while)
while (n != 0) {
    n--;
}

// 3) 범위 검사
if (0 <= x && x < 100) {
    // x가 0 이상 100 미만
}
```

4.5 논리 연산자

Heesung Oh
Copyright All rights reserved

- logical operator: &&, ||, !
  - 여러 조건식을 결합하거나 참/거짓 반전
  - 조건문(if), 반복문(while, for) 에서 사용
  - 우선 순위
    - ! 〉 && 〉 ||
  - 단락 평가 (Short-Circuit)
    - && : 왼쪽이 거짓이면 오른쪽 계산하지 않음
    - || : 왼쪽이 참이면 오른쪽 계산하지 않음

- 종류

연산자	이름	의미	예 (x:1, y:0)	결과
&&	논리 AND	두 조건이 모두 참이면 참	x && y	0 (false)
||	논리 OR	둘 중에 하나만 참이면 참	x || y	1 (true)
!	논리 NOT	조건의 참/거짓을 반전	!x	0 (false)

4.5 논리 연산자 – 예

Heesung Oh
Copyright All rights reserved

- 사용 예

```
int a = 10, b = 20;
int r1 = (a < 15 && b > 10); // 1 (참)
int r2 = (a > 15 || b > 10); // 1 (참)
int r3 = !(a == 10);         // 0 (a == 10이 참 → NOT → 거짓)
```

- 단락 평가 AND(&&)
  - a != 0 이 false 이면 (b / a) 〉 5 를 평가하지 않음

```
if (a != 0 && (b / a) > 5)
```

- 단락 평가 OR(||)
  - x == 0 이 true 이면 (b / a) 〉 5 를 평가하지 않음

```
if (x == 0 || 100 / x > 2)
```

4.5 논리 연산자 – 예

Heesung Oh
Copyright All rights reserved

- 우선 순위
  - 해석 순서: !a → b && c → !a || (b && c)

```
int r = !a || b && c;
```

- 조건 결합

```
if (score >= 80 && score < 90) {
    printf("B 등급\n");
}
```

- 조건 반전

```
if (!isFinished) {
    // 처리 수행
}
```

- 복합조건

```
if ((x >= 0 && x <= 100) || x == -1) {
    // 유효 범위 또는 특별 값 -1
```

4.6.1 비트 연산자 개요

Heesung Oh
Copyright All rights reserved

- bitwise operator (&, |, ^, ~, 〈〈, 〉〉)
  - 데이터는 byte (1byte = 8 bit) 단위로 저장
  - bit 값을 직접 조작하는 연산자
  - 플래그 처리, 하드웨어 제어, 네트워크 등 저수준 프로그래밍에서 사용

- 종류

연산자	이름	의미	예 (x:4, y:7)	결과
&	비트 AND 연산자	두 비트가 모두 1이면 1	x & y	4
|	비트 OR 연산자	두 비트 중 하나라도 1이면 1	x | y	7
^	비트 XOR 연산자	두 비트가 서로 다르면 1	x ^ y	3
~	비트 NOT 연산자	모든 비트를 반전	~x	-5
<<	왼쪽 시프트 연산자	비트를 왼쪽으로 이동	x << y	512
>>	오른쪽 시프트 연산자	비트를 오른쪽으로 이동	x >> y	0

4.6.2 비트 논리 연산자 (& | ^ ~)

Heesung Oh
Copyright All rights reserved

- 0,1에 대한 비트 논리 계산

x	y	x & y	x | y	x ^ y	~x
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

4.6.3 비트 연산자 사용 예제

Heesung Oh
Copyright All rights reserved

```
unsigned int a = 6;     // 0000 0110
unsigned int b = 10;    // 0000 1010

unsigned int r1 = a & b;    // 0000 0010  (2)
unsigned int r2 = a | b;    // 0000 1110  (14)
unsigned int r3 = a ^ b;    // 0000 1100  (12)

unsigned int r4 = ~a;
// 0xFFFFFFF9 1111 1001  (비트 반전)

unsigned int r5 = a << 1;   // 0000 1100  (12)
unsigned int r6 = b >> 1;   // 0000 0101  (5)
```

4.6.3 비트 연산자 사용 예제

Heesung Oh
Copyright All rights reserved

- 특정 비트 켜기(OR)

```
flags = flags | 0x04;    // 3번째 비트 ON
```

- 특정 비트 끄기(AND + NOT)

```
flags = flags & ~0x04;   // 3번째 비트 OFF
```

- 특정 비트 토글(XOR)

```
flags = flags ^ 0x04;    // 3번째 비트 반전
```

- 특정 비트 검사

```
if (flags & 0x04) {
    // 3번째 비트가 켜져 있음
}
```

4.6.4 시프트 연산자(shift operator)

Heesung Oh
Copyright All rights reserved

- 왼쪽 시프트(<<:Left shift)
  - 비트를 왼쪽으로 이동
  - 오른쪽에는 0이 채워짐
  - 결과적으로 2의 거듭제곱 곱셈 효과

```
x << 1   // x * 2
x << 2   // x * 4
```

- 오른쪽 시프트(>>:Right shift)
  - unsigned/ signed 에 따라 다름

- Unsigned Type Right Shift
  - 비트를 오른쪽으로 이동
  - 0으로 채움 (논리 시프트)
  - 결과적으로 2의 거듭제곱 나눗셈 효과

```
r >> 1   // x / 2
r >> 2   // x / 4
```

4.6.4 시프트 연산자(shift operator)

Heesung Oh
Copyright All rights reserved

- signed Type Right Shift
  - 부호 비트 값을 오른쪽으로 이동한 만큼 채움
  - 양수
    - 부호 비트가 0이기 때문에 2의 거듭제곱 나눗셈 효과
  - 음수
    - 2의 거듭제곱 나눗셈 효과이나 값의 시작 시 부호 비트가 1이기 때문에 1로 채우게 되어 음수 값을 유지
    - 모두 1로 비트가 채워지면 -1 값이 계속 유지

4.6.5 비트 연산자 사용 시 주의사항

Heesung Oh
Copyright All rights reserved

- signed 형 오른쪽 시프트는 구현 정의

```
signed char r = -8;    // 1111 1000
r = r >>1; // -4: 1111 1100
r = r >>1; // -2: 1111 1110
r = r >>1; // -1: 1111 1111
r = r >>1; // -1: 1111 1111 // -1 값 유지
```

- 음수 값에 대한 NOT(~) 연산
  - "~(-1)은 1인가?"

```
int x = -1;
int y = ~x;
printf("%d\n", y);
```

4.6.5 비트 연산자 사용 시 주의사항

Heesung Oh
Copyright All rights reserved

- **시프트 + 마스크에서 발생하는 문제**

```
int x = -5;            // 가정: 1111 1011
int r = (x >> 1) & 0x01;
```

- **플래그 검사 오류 예**

```
int flags = -4;   // 1111 1100 (음수는 상위 비트가 모두 1)
if (flags & 0x08)
{
  // 개발자가 의도하지 않게 여기로 들어올 수 있음
}
```

4.7.1 복합 대입 연산자의 장점

Heesung Oh
Copyright All rights reserved

- **복합 대입 연산자**
  - 연산(산술 또는 비트 연산)과 대입을 하나로 결합
  - 가독성 향상, 표현식 평가 감소, 효율성, 코드 작성의 안정성

특성	일반 대입	복합 대입
표현식	`a = a + 5;` `i = i - 1;`	`a += 5;` `i -= 1;`
가독성	`playerState->Health =` `playerState->Health - Damage;`	`playerState->Health -= Damage;`
표현식 평가	`A[index()] = A[index()] + 1;`	`A[index()] += 1;`
어셈블리	`a = a + 5;`	`a += 5;`

	`mov eax, [a]`
	`add eax, 5`
	`mov [a], eax`
		`add [a], 5`

안정성	`value = vaule + delta;  // 오타`	`value += delta;`

4.7.2 복합 대입 연산자 종류

Heesung Oh
Copyright All rights reserved

구분	연산자	의미	예 (x:8, y:5)	결과
산술	+=	더한 뒤 저장	x += y	x: 13
	-=	뺀 뒤 저장	x -= y	x: 3
	*=	곱한 뒤 저장	x *= y	x: 40
	/=	나눈 뒤 저장	x /= y	x: 1
	%=	나머지 저장	x %= y	x: 3
비트	&=	비트 AND 후 저장	x &= y	x: 0
	|=	비트 OR 후 저장	x |= y	x: 13
	^=	비트 XOR 후 저장	x ^= y	x: 13
	<<=	왼쪽으로 비트 이동 후 저장	x <<= y	x: 256
	>>=	오른쪽으로 비트 이동 후 저장	x >>=y	x: 0

4.8 증감 연산자 – 전위, 후위

Heesung Oh
Copyright All rights reserved

- Increment / Decrement operator: ++/--
  - 단항 연산자. 변수의 값 1 증가(++), 1 감소(--)

- 전위 증감
  - 변수를 먼저 증가(또는 감소), 변경된 값이 표현식의 결과

```
int a = 5;
int b = ++a;    // a는 6이 되고, b는 6
```

- 후위 증감
  - 값을 먼저 사용, 표현식 평가 과정 후 변수 값이 증가(또는 감소)

```
int a = 5;
int b = a++;    // b는 5 → 표현식 종료 후 a는 6으로 변경
```

4.8.3 포인터와 증감 연산자

Heesung Oh
Copyright All rights reserved

- 포인터가 가리키는 자료형의 크기만큼 주소 이동
  - int → 4바이트 이동
  - char → 1바이트 이동
  - 8byte double →8바이트 이동
  - 사용자 정의 구조체 → 사용자 구조체 크기만큼 이동

```
char* p1 (주소: 1000);
주소: 1000 1001 1002 1003
```

```
↑
p1
++p1 → p1 = 1001 (1 byte 이동)

int* p2 (주소: 2000);
주소: 2000      2004      2008      2012
```

```
↑
p2
++p2 → p2 = 2004 (4 byte 이동
```

4.8.4 증감 연산자 사용 시 주의사항

Heesung Oh
Copyright All rights reserved

- 변수에만 사용

```
5++;          // 오류
(a + 2)++;    // 오류
```

- 하나의 표현식에서 변수 중복 수정 금지

```
// 예시: 정의되지 않은 동작 (Undefined Behavior)
int a = 5;
// 순서점 없이 a를 여러 번 수정하고 읽음 → 결과 예측 불가능!
int b = a++ + ++a + a++ + ++a + a++ + ++a;
// 컴파일러마다, 컴파일러 버전에 따라 결과가 달라짐.
```

- 포인터의 이동 크기는 자료형 크기에 따라 결정
- a++과 a += 1은 목표는 같지만 동작 방식이 다르다

```
a += 1 : "계산 후 저장"이라는 일반 연산
a++    : "값 사용 후 1 증가"라는 단일 증감 전용 명령
```

4.8.4 증감 연산자의 동작 비교

Heesung Oh
Copyright All rights reserved

대상	연산 예	의미	결과
일반 변수	a++	값 사용 후 1 증가	a 값이 1 증가
	++a	먼저 1 증가 후 값 사용	a 값이 1 증가
	a--	값 사용 후 1 감소	a 값이 1 감소
	--a	먼저 1감소 후 값 사용	a 값이 1 감소
포인터	p++	다음 요소의 주소로 이동	sizeof(자료형) 만큼 주소 증가
	++p	먼저 다음 요소로 이동	sizeof(자료형) 만큼 주소 증가
	p--	이전 요소의 주소로 이동	sizeof(자료형) 만큼 주소 감소
	--p	먼저 이전 요소로 이동	sizeof(자료형) 만큼 주소 감소

4.9 단항 연산자

Heesung Oh
Copyright All rights reserved

- Unary Operator: +, −, !, ~, &, *
  - 피연산자가 하나
  - 부호 변환, 논리 판단, 비트 반전, 주소 접근

연산자	이름	의미
+	양수 부호	값 변화 없음. 표현을 명확히 하기 위해 사용
-	음수 부호	부호 반전(양↔음)
!	논리 NOT	0 → 1, 0이 아닌 값 → 0
~	비트 NOT	비트 반전
&	주소 연산자	변수의 메모리 주소
*	간접 참조	포인터가 가리키는 주소에 저장된 값

4.9 단항 연산자 – 주의

Heesung Oh
Copyright All rights reserved

- 틸드(tilde) 연산자
  - unsigned 정수에서 사용하는 것이 안전
  - signed 정수는 부호 비트까지 함께 반전
- 논리 NOT 연산자(!)
  - 값이 0이면 참(1), 0이 아니면 거짓(0)
- 단항 연산자 우선 순위
  - 우선순위가 높고, 결합 방향은 오른쪽에서 왼쪽

Heesung Oh
Copyright All rights reserved

- 강제 변환(explicit cast) 단항 연산자

```
double x = 3.9;
int    a = (int)x;     // 실수 → 정수, 소수점 이하 버림 → a = 3

int big  = 1000;
char c   = (char)big; // int → char, 상위 비트가 잘릴 수 있음

int n1 = 5;
int n2 = 2;

double r1 = n1 / n2;  // 정수 나눗셈 → 2 (소수점 버려짐), r1 = 2.0

double r2 = (double)n1 / n2;
// n1이 먼저 double로 변환 → 전체 연산이 실수 나눗셈
```

4.11 삼항 조건 연산자

Heesung Oh
Copyright All rights reserved

- conditional operator, ?:
  - 조건식의 결과에 따라 두 값 중 하나를 선택

- 표현식

```
조건식 ? 참일 때 값 : 거짓일 때 값
```

- 예)

```
int a = 10;
int b = 20;
int max = (a > b) ? a : b;  // max는 20
```

4.12 sizeof 연산자

Heesung Oh
Copyright All rights reserved

- "자료형 또는 표현식이 차지하는 메모리 크기(바이트 단위)"를 구하는 단항 연산자

- 표현식

```
// 자료형의 크기
sizeof(자료형)

// 변수 또는 표현식의 크기
sizeof 변수
sizeof(표현식)
```

4.12 sizeof 연산자 – 예

Heesung Oh
Copyright All rights reserved

```
printf("sizeof(char)      =>  %zu\n", sizeof(char)        );
printf("sizeof(short)     =>  %zu\n", sizeof(short)       );
printf("sizeof(int)       =>  %zu\n", sizeof(int)         );
printf("sizeof(long)      =>  %zu\n", sizeof(long)        );
printf("sizeof(long long) =>  %zu\n", sizeof(long long)   );
printf("sizeof(float)     =>  %zu\n", sizeof(float)       );
printf("sizeof(double)    =>  %zu\n", sizeof(double)      );

printf("sizeof('A')       =>  %zu\n", sizeof 'A'     );
printf("sizeof(\"Hello\") =>  %zu\n", sizeof "Hello" );
printf("sizeof(1+2)       =>  %zu\n", sizeof(1 + 2)  );
printf("sizeof(3 + 2.5)   =>  %zu\n", sizeof(3 + 2.5));
```

4.12 sizeof 연산자 – 예

Heesung Oh
Copyright All rights reserved

```
int    a = 10;
short  b = 10;
short  c = 10;
double d = 3.14;
int    arr[20]={0};
char* p = (char*)arr; // 배열 arr의 주소를 char* 포인터로 저장

printf("sizeof a      => %zu\n", sizeof a);       // 변수 a의 크기
printf("sizeof(d)     => %zu\n", sizeof(d));      // double 변수 크기
printf("sizeof(b + c) => %zu\n", sizeof(b + c));  // short 연산은 int로 승격
printf("sizeof(a + 1) => %zu\n", sizeof(a + 1));  // 표현식의 결과 자료형 크기
printf("sizeof(a == b)=> %zu\n", sizeof(a == b)); // 관계 연산 결과는 int
printf("sizeof(arr)   => %zu\n", sizeof(arr));    // 배열 전체 크기
printf("sizeof(arr[0])=> %zu\n", sizeof(arr[0])); // 배열 요소 1개의 크기
printf("sizeof(p)     => %zu\n", sizeof(p));      // 포인터의 크기
printf("sizeof(*p)    => %zu\n", sizeof(*p));     // p가 가리키는 자료형의 크기
printf("array elements => %zu\n", sizeof(arr) / sizeof(arr[0])); // 배열 길이
```

4.13 함수 호출 연산자

Heesung Oh
Copyright All rights reserved

- function call operator, ( )
  - 함수 이름 뒤 괄호 ( )를 붙여 함수를 실행

- 표현식

```
함수이름(인자1, 인자2, ...)
```

- 예

```
// sum(int, int) 함수의 주소를 fp에 저장
int (*fp)(int, int) = sum;

// fp는 sum 함수의 주소이므로, 호출 방식은 sum과 동일
fp(3, 4);   // sum(3, 4)와 같음
```

4.14 배열 인덱스 연산자

Heesung Oh
Copyright All rights reserved

- array subscript operator, [ ]
  - 배열의 특정 요소(element) 접근

```
int arr[5] = {10, 20, 30, 40, 50, 60};
int v = arr[3];    // 40, *(arr + 3)
```

4.15 구조체 멤버 접근 연산자

Heesung Oh
Copyright All rights reserved

- member access operator, . , ->
  - 구조체 멤버 접근

구조체변수.멤버	포인터->멤버

- 점 연산자 (dot operator, .)
  - 구조체 변수를 통해 멤버에 접근

```
struct Point { int x, y; };
struct Point p = {10, 20};
int a = p.x;    // 10
```

- 화살표 연산자 (arrow operator, ->)
  - 구조체 포인터를 통해 멤버에 접근
  - ptr->m은 (*ptr).m과 동일

```
struct Point p = {10, 20};
struct Point* pp = &p;
int b = pp->y;    // 20
```

4. 학습 정리

Heesung Oh
Copyright All rights reserved

- 연산자
  - 값의 계산, 비교, 논리 판단, 대입을 표현하는 문법적 기호
- 대입 연산자(=)
  - 오른쪽 표현식의 결과를 왼쪽 변수에 저장
  - 값의 형태는 항상 왼쪽 변수의 자료형에 의해 결정
- 쉼표(,) 연산자
  - 여러 표현식을 왼쪽에서 오른쪽 순서로 평가, 마지막 표현식의 결과 반환
- 산술 연산자
  - 정수와 실수 계산에 사용
  - 정수 나눗셈은 소수점 이하가 버려지며, 모듈로(%)는 정수에서만 사용된다.
- 비트 연산자
  - 비트 단위 연산 수행. 플래그 처리와 마스크 연산에 사용
- 복합 대입 연산자
  - 연산과 대입을 결합하여 표현
- 단항 연산자
  - 하나의 피연산자에 대해 부호 변경, 논리 반전, 주소 취득, 간접 참조, 증감 연산 등을 수행
- 함수 호출 연산자
  - 함수를 실행하며 반환값이 표현식의 결과
- 인덱스 연산자[]
  - 임의 접근연 산자
- 구조체 멤버 접근 연산자(. →)
  - 구조체의 멤버 접근 연산자

4. 학습 정리 – 연산자 우선 순위

Heesung Oh
Copyright All rights reserved

우선순위	연산자 범주	연산자 기호	결합 방향
1	최우선 (Primary)	(), [], ->, ., (type){list}	L → R
2	단항 (Unary)	++, -- (후위), ++, -- (전위), +, -, !, ~, (type), *, &, sizeof, _Alignof	R → L
3	산술 (Multiplicative)	*, /, %	L → R
4	산술 (Additive)	+, -	L → R
5	비트 이동 (Shift)	<<, >>	L → R
6	관계 (Relational)	<, <=, >, >=	L → R
7	관계 (Equality)	==, !=	L → R
8	비트 논리 (AND)	&	L → R
9	비트 논리 (XOR)	^	L → R
10	비트 논리 (OR)	|	L → R
11	논리 (AND)	&&	L → R
12	논리 (OR)	||	L → R
13	조건 (Ternary)	?:	R → L
14	대입 (Assignment)	=, +=, -=, *=, /=, %=, <<=, >>=, &=, ^=, |=	R → L
15	쉼표 (Comma)	,	L → R

4. 실습 과제

Heesung Oh
Copyright All rights reserved

- 산술 연산자 실습
- 증감 연산자 실습
- 복합 대입 연산자 실습
- 단항 연산자 실습
- 형 변환(cast) 실습
- sizeof 실습
- 삼항 조건 연산자 실습